

DEVOPS CAREER SIMULATOR

PROJECT 02

Northwind Design Studio

Full Technical Documentation — Concepts, Commands, and Every Incident Diagnosed Along the Way

Prepared for Johnson · DevOps Career Simulator

1. What Was Built

Northwind Design Studio started as the FTP-dragging, deploy-breaks-weekly scenario from the project brief: three designers, one developer, no version control, no automated deploys. By the end of this documented stretch of work, the following was actually running, live, on real infrastructure:

- An Express.js application with a real frontend (SaaS-style marketing site) and a real backend (routes, middleware, tests)
- Google OAuth login (via Passport.js) and a Stripe test-mode checkout flow
- A real domain (`northwind-design.online`) purchased on Namecheap, DNS delegated to Route 53
- Nginx running as a reverse proxy in front of the Node app, with Let's Encrypt HTTPS (auto-renewing)
- A second, fully independent **staging** environment on its own EC2 instance, own subdomain, own certificate — access-restricted via nginx basic auth, so it's testable by the team without being open to the public internet
- A GitHub Actions CI/CD pipeline: `test → deploy-staging → deploy (production, with manual approval gate)`
- `pm2` process management on both servers, configured to survive reboots
- Trunk-based git workflow in active use — multiple short-lived feature branches, all merged promptly

This document exists because a huge amount of the real learning happened in the debugging, not the tutorial steps. Below is every distinct problem encountered, how it was actually diagnosed (not just the fix), and the underlying pattern worth keeping.

Still deliberately open, not yet done: SSH hardening beyond the key itself (`fail2ban` , narrowing the security group to GitHub's published runner IP ranges) was discussed but not yet executed — the key alone is a real, legitimate boundary for now, but this is flagged honestly as a next step rather than treated as finished.

2. Concepts Covered

Git & Version Control

The save-system analogy: repository = save file with full history, commit = a save point, branch = a parallel save slot. Core loop: `status` → `add` → `commit` → `push/pull`. `log --oneline` to see history with short hashes. `checkout <hash>` to view an old commit (detached HEAD — safe, nothing lost). `revert <hash>` creates a new commit that undoes a specific old one — safe on shared branches, history stays intact. `reset` physically rewrites history — dangerous once others have pulled. `stash` shelves uncommitted changes with no commit and no message, for exactly the "urgent interruption" scenario. Branching models: trunk-based (short-lived branches, fast merges — chosen for Northwind's 4-person team), git-flow (heavier, staged branches for scheduled releases), GitHub flow (branch → PR → merge → deploy).

GitHub Actions / CI-CD

A workflow file lives at `.github/workflows/*.yaml`, triggered by `on:`. A job runs on a fresh, empty virtual machine (`runs-on: ubuntu-latest`) — nothing pre-installed, not even your code. `uses:` runs a prebuilt, reusable action (e.g. `actions/checkout` to pull your repo onto that empty machine); `run:` executes a raw shell command yourself. `needs:` chains jobs so one only runs after another succeeds. `if: github.ref == 'refs/heads/main'` restricts a job to a specific branch, since `on: push` alone fires on every branch. `environment: production` plus a configured "required reviewers" rule in repo Settings makes a job pause and wait for a human's manual approval before running — the actual mechanism behind a promotion gate from staging to production. GitHub Secrets store credentials encrypted, referenced as `${{ secrets.NAME }}`, never appearing in plaintext in the file or logs.

Nginx & Reverse Proxies

Nginx as a reverse proxy doesn't answer requests itself — it forwards them to a running app process (`proxy_pass http://localhost:3000`) and relays the response back. This matters because ports below 1024 (including 80 and 443) are privileged, requiring root — nginx handles that binding safely and hands off to your unprivileged Node process on 3000. `server_name` lets one nginx instance host multiple domains, each with its own `server {}` block. `proxy_set_header Host / X-Real-IP` pass the real requested domain and visitor IP through to the app, which otherwise would only see `localhost` traffic.

HTTPS / Let's Encrypt

`certbot --nginx` auto-detects a matching `server_name` block and both issues a certificate and rewrites the nginx config to add a 443 block plus an HTTP→HTTPS redirect. It needs a working config to attach to *before* it runs — running it against a domain with no matching nginx block causes it to silently fall back to editing the default site instead. Certificates renew automatically via a systemd timer (`certbot.timer`), no manual action needed every 90 days.

Process Management

`pm2` keeps a Node process running in the background and restarts it on crash — solves the problem of an app dying the moment an SSH session closes. `pm2 save` snapshots the current process list; `pm2 startup` registers pm2 itself as a system service so both the app and pm2 survive a full server reboot, not just a disconnect.

OAuth & Third-Party Auth

Passport.js delegates authentication entirely to Google/Facebook — no local password storage. Each OAuth provider requires a registered app with an exact, matching redirect/callback URI; a mismatch (wrong path, wrong protocol, wrong port) causes a silent or explicit rejection. Separate credentials were used for staging vs. production so a leak in one environment can't compromise the other.

Environments & Staging

Staging is a completely separate deploy target — same codebase, different infrastructure (its own EC2 instance, subdomain, cert, process) — never a git branch. The point is to test the exact real deploy mechanics somewhere disposable before they touch production.

Express's Dual Role — "Frontend" and "Backend" Aren't Two Apps

Express isn't two separate applications wearing different hats — it's **one process, doing two different jobs** depending on what a request asks for. When a browser requests a plain file (a homepage, a CSS file), `express.static` hands it over exactly as-is — behaving like nginx did for the café's static site. When a request needs actual logic (checking a session, validating a form, calling Stripe), Express runs real code and returns a computed answer, often JSON, not a file at all. "Frontend" and "backend" here are two *behaviors* of the same server, not two programs.

Multiple Pipelines: One App, Many Services

A single app (a monolith) can have more than one workflow file — e.g. a full `ci.yml` for normal changes, and a second, leaner pipeline scoped to a specific branch pattern for genuine emergencies. In a microservices system, each service typically gets its own independent pipeline entirely (its own repo, or a `paths:` filter in a monorepo so a change to one service doesn't retrigger or block deploys of unrelated ones). Neither requires literally duplicating infrastructure — often a smarter trigger condition on the *same* pipeline achieves the same goal more simply than a second file would.

Additional GitHub Actions Triggers (beyond `push`)

`pull_request` — runs checks the moment a PR opens or updates, before anything merges; this is the more common real-world pattern than testing after a push, since it surfaces red/green directly on the PR and can be required before the merge button even unlocks. `workflow_dispatch` — adds a manual "Run workflow" button in the Actions tab, for anything that shouldn't fire automatically. `schedule` — cron-based, runs on a timer regardless of any code change (nightly jobs, scheduled cleanup). Multiple triggers can be combined under one `on:` block.

3. Build Timeline — How Northwind Was Actually Built, Step by Step

The Incident Log explains what broke. This section explains the actual order things were built in — the parts that worked cleanly the first time don't show up in an incident log, but they're still real steps worth having a record of.

3.1 — Initial Scaffold

The very first Claude Code prompt asked for a minimal Express starter: a couple of routes (home, portfolio, contact), a `tests/` folder with basic Jest+supertest tests hitting each route for a 200 status, a `.gitignore`, and — critically — a `CLAUDE.md` file in the repo root establishing a standing guardrail: Claude Code may explain errors and suggest fixes without rewriting full configs or infrastructure on the user's behalf, and must never make git, branching, or CI/CD decisions unilaterally. This file is what kept every later scaffolding prompt honest about staying out of the graded, hands-on work.

3.2 — First Backend Attempt: Local Auth (later fully replaced)

Before OAuth existed in this project, an earlier version used local authentication: `bcrypt` for password hashing, `jsonwebtoken` for session tokens, and a plain in-memory array standing in for a database (deliberately not a real DB yet — same "simple now, correct later" pattern as other shortcuts in this project). Routes: `POST /signup`, `POST /login`, a JWT-protected `GET /dashboard`. This entire approach was later **fully superseded** by Google OAuth via Passport.js — no local password storage survived into the final version. Worth knowing this existed, since early exploration of a codebase can turn up old, now-irrelevant auth code if not careful.

Note: if this earlier local-auth code is ever found still present in the repo, it's dead code from an earlier iteration, not a second working auth system — the OAuth version is the real one.

3.3 — First Deploy, the "Old Way" (deliberate, before git/CI-CD existed)

Before any automation existed, the app was deployed by hand once, on purpose, specifically to feel the pain that git and CI/CD later solve:

```
rsync -avz -e "ssh -i yourkey.pem" \
  --exclude 'node_modules' \
  --exclude '.git' \
  --exclude '.env' \
  ./ \
  ubuntu@your-ec2-ip:~/northwind-design-studio/
```

`-avz` — archive mode (preserves file structure/permissions), verbose output, compressed transfer. `-e "ssh -i ..."` — tells rsync to tunnel over SSH using the existing key, same authentication already used for direct SSH. `--exclude` flags skip `node_modules` (huge, reinstall fresh on the server instead), `.git` (not needed to run the app), and `.env` (secrets shouldn't travel this casually). After copying, `npm install` was run *fresh on the server*, then `npm start` — and the very first real lesson landed immediately: the app died the instant the SSH session closed, since nothing was keeping it alive in the background. That specific pain is exactly what `pm2` solves later.

3.4 — Domain Purchase & Route 53 Delegation

The domain (`northwind-design.online`) was bought on Namecheap, not Route 53 directly — meaning Namecheap is the **registrar** and Route 53 needed to become the **DNS host**, two different roles that don't have to be the same company. Steps: (1) Route 53 → Create hosted zone for the exact domain name; (2) copy the 4 auto-generated NS (nameserver) records; (3) in Namecheap → Domain List → Manage → Nameservers, switch from "Namecheap BasicDNS" to "Custom DNS" and paste in those exact 4 AWS nameservers — this is the actual delegation, telling the internet "for this domain, ask AWS, not Namecheap"; (4) wait for propagation (usually well under the 48-hour worst case); (5) once propagated, create an A record in the Route 53 hosted zone pointing at the EC2 instance's public IP.

3.5 — The Real Backend: OAuth + Stripe

The local-auth version was replaced entirely with a proper prompt: Passport.js with Google (and briefly Facebook, later removed) OAuth strategies, `express-session` to persist login state, protected `/dashboard`, and a Stripe Checkout integration in **test mode only** — a route creating a Checkout Session for whichever pricing tier was selected, redirecting to Stripe's hosted checkout page, with `/checkout/success` and `/checkout/cancel` landing routes. All credentials were required to come from `process.env`, never hardcoded, with a `.env.example` listing placeholder variable names only.

GOOGLE CLOUD CONSOLE SETUP, DONE MANUALLY (CLAUDE CODE CANNOT DO THIS STEP)

APIs & Services → Credentials → OAuth client ID → Web application. **Authorised JavaScript origins:** the bare domain, e.g. `https://northwind-design.online` — governs which origins can initiate the OAuth flow via browser JS. **Authorised redirect URIs:** the exact callback path, e.g. `https://northwind-design.online/auth/google/callback` — must match the app's real route character-

for-character, or Google rejects the redirect. A **separate** OAuth client (different credentials, different redirect URI) was later registered for staging specifically, so a leak in one environment can't compromise the other.

3.6 — Staging Environment: Full Build, Step by Step

Staging was built as an entirely independent environment, not a shortcut:

1. Launch a second EC2 instance from scratch (same process as the very first one)
2. `sudo apt update` — refresh the package index before installing anything
3. Create a `staging` A record in Route 53, pointing at this new instance's IP — a subdomain, not a new domain purchase (DNS records under an already-owned domain are free)
4. Install nginx, write a plain-HTTP reverse-proxy config for `staging.northwind-design.online`, enable and reload it — *before* running certbot (getting this order backwards is what caused incident B3)
5. Install certbot (`sudo apt install certbot python3-certbot-nginx -y`) and run it against the staging subdomain specifically
6. Clone the repo fresh onto this box (not `rsync` 'd, since staging needed real git history for the eventual CI/CD deploy to `git pull` against)
7. Create a fresh `.env` by hand on this server, with staging-specific values: its own `APP_BASE_URL`, its own freshly-generated `SESSION_SECRET` (`openssl rand -hex 32`, never reused from production), and the separate staging OAuth credentials
8. Install `pm2` globally (`sudo npm install -g pm2` — it's an npm package, not an apt package, a real mix-up encountered live), start the app under a clearly distinct name (`northwind-staging`, never just `northwind` — the exact naming mismatch that caused incident E4)
9. Restrict access with nginx basic auth (`htpasswd`), since staging shouldn't be reachable by the public internet the way production is

3.7 — Production Approval Gate: Settings Steps

The `environment: production` line in `ci.yml` only pauses a job if a matching protection rule actually exists. Steps taken: GitHub → repo → Settings → Environments → New environment → name it exactly `production` (must match the YAML reference character-for-character) → enable "Required reviewers" → add the account as an approver → save. Once configured, any job referencing this environment sits in a "Waiting" state with a visible "Review deployments" button until a human explicitly approves it — a genuine human-in-the-loop gate, not a simulated one.

4. Incident Log

Every entry follows the same shape on purpose: **what you saw**, **how the actual cause was located** (not guessed), **what was actually wrong**, and **the fix**. The diagnostic column is the part worth re-reading — it's the transferable skill.

A. Application & Server Basics

A1 — MODULE_NOT_FOUND on first server start

SYMPTOM

Node crashed immediately on `npm start`, error trace pointing at `server.js`, code `MODULE_NOT_FOUND`.

HOW IT WAS FOUND

Asked what was on line 1 of `server.js`, and whether `npm install` had run before `npm start` — the two most likely causes for a missing-module error on a freshly copied project.

ROOT CAUSE

Dependencies were never installed — `node_modules` didn't exist yet.

FIX

```
npm install
npm start
```

PATTERN

Any "module not found" on a fresh checkout/copy of a Node project is almost always a missing `npm install`, checked before anything more exotic.

A2 — Domain loading nginx's default page instead of the app (production)

SYMPTOM

`http://northwind-design.online` unreachable / not resolving as expected during initial reverse-proxy setup.

HOW IT WAS FOUND

Layered check: `curl localhost:3000` (is the app itself alive?), `systemctl status nginx` + `nginx -t` (is nginx healthy and its config valid?), each layer tested independently before assuming which one was broken.

ROOT CAUSE

The app simply hadn't been started on the server — `npm install` / `npm start` were never run there, only locally.

RUN `npm install` AND `npm start` ON THE ACTUAL SERVER, NOT JUST LOCALLY.

PATTERN

Nginx being perfectly configured is irrelevant if there's nothing listening on the port it's forwarding to. Always confirm the upstream app is alive first.

4. Incident Log (continued)

B. Domain, DNS & HTTPS

B0 — Nginx config corrupted on paste (markdown auto-conversion)

SYMPTOM

After pasting a reverse-proxy config into `nano`, the `server_name` line showed a broken, markdown-style link fragment — `[www.north](https://www.north)>` — instead of a plain domain name, with the line's terminating semicolon missing entirely.

HOW IT WAS FOUND

Read the pasted content back before saving — comparing what was actually in the editor against what should have been there, rather than saving on faith.

ROOT CAUSE

The source the config was copied from auto-converted plain URLs into markdown link syntax (`[text](url)`); pasting that directly into a terminal editor carried the broken formatting straight into the config file.

CANCELLED THE SAVE (`CTRL+C` AT THE WRITE PROMPT), DELETED THE CORRUPTED LINE INSIDE THE EDITOR, AND RETYPED THE LINE MANUALLY RATHER THAN PASTING IT A SECOND TIME — CONFIRMED WITH `CAT` BEFORE SAVING/ENABLING THE SITE.

Warning: content copied from chat interfaces, docs sites, or notes apps can carry invisible formatting (smart quotes,

markdown link syntax, non-breaking spaces) that looks identical on screen but breaks a config file's syntax. Always `cat` a file back after pasting into it, before trusting it's correct.

B1 — Certbot failed: NXDOMAIN for www subdomain

SYMPTOM

`certbot --nginx -d northwind-design.online -d www.northwind-design.online` failed with a DNS problem specifically for the `www` domain.

HOW IT WAS FOUND

Certbot's own error message named the exact failing domain and reason (`NXDOMAIN looking up A for www...`) — read directly, not guessed.

ROOT CAUSE

Only an A record for the bare domain existed in Route 53; `www` had never been created at all.

ADDED A SECOND A RECORD FOR `www` , SAME IP, THEN RE-RAN THE IDENTICAL CERTBOT COMMAND.

PATTERN

Every hostname in a certbot `-d` list must independently resolve — the tool validates each one separately, and one failing domain fails the whole run.

B2 — HTTPS unreachable despite valid nginx config and running services

SYMPTOM

Both `nginx -t` and `systemctl status nginx` looked healthy, but `curl https://northwind-design.online` from the server itself returned `Failed to connect ... port 443`.

HOW IT WAS FOUND

Two-step isolation: `ss -tlnp | grep 443` to confirm nginx was actually bound to the port at all, and `curl -Ik https://localhost` to test HTTPS bypassing DNS/the internet entirely. Both pointed away from nginx itself and toward the network boundary.

ROOT CAUSE

The EC2 security group had never had an inbound rule added for port 443 — only 80 and 22 existed.

ADDED AN INBOUND RULE: TCP 443, SOURCE 0.0.0.0/0, ON THE CORRECT ATTACHED SECURITY GROUP.

PATTERN

A perfectly correct application-layer setup (nginx, certs) is invisible to the outside world if the network layer (security group) never opens the door. Always check both layers independently, especially after adding a new port to an existing service.

B3 — Certbot silently modified the wrong file (staging)

SYMPTOM

After running certbot for `staging.northwind-design.online`, the browser showed nginx's generic "Welcome to nginx!" page instead of the app, despite certbot reporting success.

HOW IT WAS FOUND

Re-read certbot's own output line closely: `"Successfully deployed certificate ... to /etc/nginx/sites-enabled/default"` — not the expected staging-specific file.

ROOT CAUSE

Certbot was run *before* a proper `server_name staging...` block existed anywhere. With no matching block to attach to, it fell back to nginx's default catch-all site and bolted the SSL config onto that instead.

WROTE THE PROPER STAGING SITE FILE FIRST, SYMLINKED IT INTO `SITES-ENABLED` , THEN RE-RAN CERTBOT — THIS TIME IT FOUND THE CORRECT BLOCK. THE ACCIDENTAL EDITS TO `DEFAULT` WERE MANUALLY STRIPPED BACK OUT AFTERWARD.

PATTERN

Certbot's `--nginx` plugin needs a real, matching config to exist *before* it runs — order matters, and it fails silently into the wrong file rather than erroring loudly.

B4 — Duplicate default_server conflict (staging, after cleanup)

SYMPTOM

`nginx -t` failed outright: `a duplicate default server for 0.0.0.0:80 in .../staging.northwind-design.online:2`.

HOW IT WAS FOUND

Rather than guessing, both the `default` and `staging` config files were printed and compared side by side, in full.

ROOT CAUSE

A copy-paste mistake — the `staging` config file had been overwritten with the exact same content as `default` (including

`default_server` and `server_name _;`), rather than the actual staging-specific proxy config.

REWROTE THE STAGING FILE FROM SCRATCH (VIA VIM, USING GGDG TO CLEAR IT FULLY FIRST), WITH THE CORRECT SERVER_NAME STAGING... AND PROXY_PASS CONTENT, THEN RE-VERIFIED BY CAT -ING IT BACK BEFORE TESTING.

PATTERN

When two files should be different but a config error implies they're colliding, don't assume — read both files' actual contents. A confusing error message is often a simple, mechanical mistake (wrong paste target) wearing a scary costume.

C. Git, GitHub & Environment Files

C1 — .env.example silently ignored by git

SYMPTOM

Claude Code was asked to create `.env.example`; it never appeared in `git status` as untracked.

HOW IT WAS FOUND

Read `.gitignore` directly rather than assuming the file was never created.

ROOT CAUSE

A wildcard rule, `.env.*`, matched and silently excluded `.env.example` along with the real `.env`.

```
echo '!.env.example' >> .gitignore
```

The `!` negates the broader rule above it, carving out one exception.

PATTERN

A file that "should" be tracked but isn't showing up is a `.gitignore` question before it's anything else.

C2 — Bash history-expansion error writing the gitignore fix

SYMPTOM

`echo "!.env.example" >> .gitignore` failed with `bash: !.env.example: event not found`.

HOW IT WAS FOUND

Recognized the error as bash's history-expansion feature, triggered specifically by `!` inside *double* quotes.

ROOT CAUSE

Double quotes don't protect `!` from bash interpreting it as a history reference.

USED SINGLE QUOTES INSTEAD: `ECHO '!.ENV.EXAMPLE' >> .GITIGNORE`.

PATTERN

Single quotes in bash are fully literal; double quotes still allow some special-character interpretation (`!`, `$`, backticks).

C3 — Accidental revert of the wrong commit

SYMPTOM

While practicing `git revert`, the editor showed "Revert 'initial'" — the entire original project scaffold — instead of a small one-file practice commit.

HOW IT WAS FOUND

Read the nano commit-message editor's content carefully before saving, and checked `git log --oneline` afterward to see exactly what had actually landed.

ROOT CAUSE

The wrong commit hash was used with `git revert`. An attempted `Ctrl+X / N` abort didn't actually cancel it as expected — the revert of the scaffold commit went through anyway.

CONFIRMED THE MISTAKE HADN'T BEEN PUSHED YET (ORIGIN/MAIN WAS STILL ON THE OLD COMMIT), THEN RAN GIT REVERT AGAIN — THIS TIME ON THE ACCIDENTAL REVERT COMMIT ITSELF — RESTORING THE REAL SCAFFOLD, VERIFIED WITH LS AND NPM TEST BEFORE FINALLY PUSHING.

PATTERN

Always double-check the exact hash before reverting, and never push immediately after a risky operation — verify locally first. A revert is itself revertible, which is exactly why it's the safe choice over `reset`.

C4 — OAuth login entry point missing from the UI

SYMPTOM

Backend OAuth routes worked, but there was no visible "Login with Google" button anywhere reachable.

HOW IT WAS FOUND

Traced back to the original Claude Code scaffolding prompt — it specified the routes and protected `/dashboard`, but never specified *where in the UI* a human should click to reach them. Confirmed by checking `dashboard.ejs`: a logout link existed there, but no login link anywhere reachable pre-login.

ROOT CAUSE

A genuine gap in the original instructions, not a Claude Code error — a classic chicken-and-egg bug (the only login link lived on the page that required already being logged in).

A FOLLOW-UP, NARROWLY-SCOPED CLAUDE CODE PROMPT TO ADD A VISIBLE LOGIN LINK TO THE HOMEPAGE NAV SPECIFICALLY.

PATTERN

Backend correctness and UI reachability are separate concerns — a fully working route is useless if nothing in the interface links to it.

C5 — APP_BASE_URL misconfigured

SYMPTOM

Caught during a routine `.env` review, before it caused a failure.

HOW IT WAS FOUND

Reading the code comment above the field ("*used to build OAuth/Stripe callback URLs*") and comparing it against the actual pasted value.

ROOT CAUSE

The full Google callback URL had been pasted into a field meant to hold only the bare domain — every other URL built from it would have been malformed.

CORRECTED TO JUST `HTTPS://NORTHWIND-DESIGN.ONLINE`, NO PATH, NO TRAILING SLASH.

PATTERN

Read what a config comment says the value is *for*, not just what looks plausible — a value can be a "real" URL and still be the wrong one for that specific field.

D. Security Groups & Access

D1 — Invalid security group source ("CIDR block ... has to be specified")

SYMPTOM

AWS console rejected a bare IP address entered as a security group source.

HOW IT WAS FOUND

Read the console's own validation message directly.

ROOT CAUSE

AWS requires CIDR notation, not a bare IP — even for a single address.

APPENDED `/32` (MEANING "EXACTLY THIS ONE ADDRESS") — E.G. `197.211.53.99/32`.

PATTERN

`/32` for one IP, wider ranges for anything broader — security group sources are always CIDR blocks, never bare addresses.

4. Incident Log (continued)

E. The CI/CD Pipeline Saga

This was the largest single debugging arc of the whole project — five distinct, unrelated failures in sequence, each one only revealed once the previous was fixed. Worth reading end to end: it's a realistic picture of what "getting a pipeline working" actually looks like in practice.

E1 — Malformed STAGING_HOST secret

SYMPTOM

`deploy-staging` failed instantly: `dial tcp: lookup tcp:///staging.northwind-design.online: unknown port`.

HOW IT WAS FOUND

The error string itself was the clue — three slashes is not a normal hostname, indicating a URL scheme or stray character had been baked into the secret's value.

ROOT CAUSE

The `STAGING_HOST` secret held something other than a bare hostname (likely a full URL with `https://` or a trailing slash).

```
gh secret set STAGING_HOST --body "staging.northwind-design.online"
```

PATTERN

Secrets can't be viewed once set — if a connection error looks like malformed input rather than a network problem, reset the secret cleanly rather than guessing what's currently in it.

E2 — SSH i/o timeout from the GitHub Actions runner

SYMPTOM

Next run, different error: `dial tcp <ip>:22: i/o timeout`.

HOW IT WAS FOUND

Recognized the error type as "reached the network layer, got no response" (distinct from an auth error, which would mean the connection succeeded). Cross-referenced against a security decision made earlier in the same session: staging's SSH had been deliberately restricted to one specific personal IP.

ROOT CAUSE

GitHub Actions runners connect from GitHub's own cloud IP ranges, not the developer's personal IP — the earlier IP-restriction, correct for a human, was blocking the pipeline itself.

OPENED PORT 22 MORE BROADLY ON STAGING'S SECURITY GROUP, ACCEPTING THAT THE SSH KEY ITSELF — NOT THE IP — IS THE REAL ACCESS CONTROL FOR THIS LOW-STAKES ENVIRONMENT.

PATTERN

A security decision that's correct for one caller (a human) can silently break a different, legitimate caller (an automated pipeline) that doesn't share the same network origin.

E3 — SSH authentication failure ("no supported methods remain")

SYMPTOM

Connection reached the server this time, but was rejected: `ssh: handshake failed: ... no supported methods remain`.

HOW IT WAS FOUND

Distinguished from E2 by the error category — this is authentication, not network reachability, meaning the previous fix worked and a new, different layer had failed.

ROOT CAUSE

The `STAGING_SSH_KEY` secret didn't match the actual key pair assigned to the staging EC2 instance at launch time.

CHECKED THE INSTANCE'S ACTUAL "KEY PAIR NAME" IN THE EC2 CONSOLE, THEN RESET THE SECRET FROM THE CORRECT .PEM FILE:

```
gh secret set STAGING_SSH_KEY < ~/.ssh/correct-key.pem
```

PATTERN

Piping a key file directly into a secret (`<` redirect) avoids copy-paste corruption — but only helps if it's the *right* file to begin with. Always verify which key pair an instance actually expects.

E4 — pm2 process name mismatch

SYMPTOM

SSH succeeded, `git pull` and `npm install` both worked, then: `[PM2][ERROR] Process or Namespace northwind not found`.

HOW IT WAS FOUND

Read the full, successful portion of the log first to confirm exactly where the chain actually broke, then compared the failing command's process name against the name actually used when the process was first started, weeks-worth-of-messages earlier in the session.

ROOT CAUSE

The workflow script said `pm2 restart northwind`, but the staging process had been started as `northwind-staging` — a simple naming mismatch between two different points in the session.

CORRECTED THE SCRIPT LINE TO USE `NORTHWIND-STAGING`, AND ADDED A FALLBACK PATTERN FOR RESILIENCE:

```
pm2 restart northwind-staging || pm2 start server.js --name northwind-staging
```

PATTERN

`pm2 restart` only works on a process that already exists by that exact name — the `||` fallback makes a deploy script idempotent, working whether or not the process happens to already be running.

E5 — pm2 process genuinely not running anymore

SYMPTOM

Same error again, correct process name this time: `Process or Namespace northwind-staging not found`.

HOW IT WAS FOUND

SSH'd in directly and ran `pm2 list` — the process was simply gone, not misnamed.

ROOT CAUSE

The pm2 process hadn't been told to persist — it didn't survive some combination of earlier SSH disconnects/instance activity during the long debugging session.

```
pm2 start server.js --name northwind-staging
pm2 save
sudo env PATH=$PATH:/usr/bin /usr/local/lib/node_modules/pm2/bin/pm2 startup systemd -u ubuntu --hp /home/ubuntu
```

`pm2 save` snapshots the process list; `pm2 startup` registers pm2 as a real system service so both survive a full reboot, not just a disconnect.

PATTERN

`pm2 start` alone only survives until the next reboot or daemon restart — `save` + `startup` together are what make a process genuinely persistent.

E6 — pm2 start run from the wrong directory

SYMPTOM

`[PM2][ERROR] Script not found: /home/ubuntu/server.js`.

HOW IT WAS FOUND

`ls` immediately showed the actual folder structure — the app lived one directory level down.

ROOT CAUSE

`pm2 start server.js` was run from the home directory, not from inside the app's own folder.

CD INTO THE CORRECT FOLDER FIRST, THEN RE-RAN THE SAME COMMAND.

PATTERN

`pm2 start` resolves script paths relative to the current working directory — always confirm you're standing in the right place first.

E7 — pm2 EADDRINUSE port conflict

SYMPTOM

Process started successfully, then immediately showed status `errored` in `pm2 list`.

HOW IT WAS FOUND

`pm2 logs northwind-staging --lines 30 --nostream` — the actual crash trace named the real cause directly: `EADDRINUSE ... port 3000`. A second pm2 process, auto-named `server`, was found already running via `pm2 list`.

ROOT CAUSE

An earlier, ad-hoc `pm2 start server.js` (without an explicit `--name`) had left a duplicate copy of the same app running under a different auto-generated name, already holding port 3000.

```
pm2 delete server
pm2 restart northwind-staging
pm2 save
```

PATTERN

Two processes can't bind the same port — a "crash loop" immediately after a clean start is a strong signal to check for a duplicate/leftover process, not a code bug.

E8 — Ghost "stuck queued" GitHub Actions run

SYMPTOM

A workflow run sat at "Queued" for over 40 minutes; the Cancel button in the UI failed with "failed to cancel."

HOW IT WAS FOUND

Ruled out causes methodically rather than guessing: checked Actions billing usage (clean, \$0.08 — not a spending limit), used `gh run list` / `gh run cancel` via CLI instead of the web UI (CLI reported the run as already "completed," revealing a display-sync lag between the VS Code extension panel and GitHub's actual backend). Established the web UI at github.com, hard-refreshed, as the trustworthy source of truth.

ROOT CAUSE

A stale/laggy status display in one interface (VS Code's Actions panel), not a genuine backend problem.

IGNORED THE GHOST RUN ENTIRELY AND TRIGGERED A CLEAN, FRESH RUN:

```
git commit --allow-empty -m "clean retrigger"
git push
```

This surfaced the real, next, legitimate error (E4 above).

PATTERN

When two different tools disagree about the same state, trust the platform's own primary interface (or its CLI) over a third-party integration's display — and don't burn time debugging a symptom that turns out to be a UI sync issue rather than a real failure.

E9 — SSH i/o timeout, production (same bug, different box)

SYMPTOM

`deploy` (production job) failed after `test` and `deploy-staging` both passed cleanly: `dial tcp <prod-ip>:22: i/o timeout`.

HOW IT WAS FOUND

Recognized as identical in shape to E2 (same error text, same layer) — checked production's own security group for the same restriction that had caused staging's version of this bug.

ROOT CAUSE

Production's SSH rule was also locked to a personal IP rather than allowing GitHub Actions' runner range — same class of mistake as E2, just not yet applied to the second box.

OPENED PORT 22 ON PRODUCTION'S SECURITY GROUP THE SAME WAY AS STAGING'S FIX.

PATTERN

A fix applied to one environment doesn't automatically apply to a twin environment — staging and production are separate infrastructure with separate security groups, even when the mistake behind them is identical. This is exactly why the pipeline itself deploys to both independently, rather than assuming "fixed once" means "fixed everywhere."

Pipeline status: fully green. `test` → `deploy-staging` → `deploy` now completes end to end, with the production job correctly pausing for manual approval before it runs. Nine real, independent bugs — across DNS, nginx, git, environment config, security groups, and process management — diagnosed and fixed in a single session, entirely from reading actual error output rather than guessing.

F. The Hotfix Pipeline & Dark Mode Saga

This section is longer than the others on purpose. Most of what went wrong here wasn't a technical bug at all — it was *process*: git operations run in the wrong order, conclusions drawn before verifying, one real root-cause bug hiding underneath three false alarms. Documented in full, in the order it actually happened, because the sequence itself is the lesson.

F1 — Misconception: do git branches nest, like a "master hotfix" with sub-branches?

THE QUESTION

Whether `hotfix/dark-mode` was a child of some parent `hotfix` branch.

THE CORRECTION

Git branches are flat. There is no real parent-child relationship implied by a slash in a branch name — `hotfix/dark-mode` is not "inside" a branch called `hotfix`. The slash is purely a naming convention for humans (and for pattern-matching tools like `startsWith`); git's actual history graph only tracks commit-to-commit parent links, never branch-to-branch relationships. Each `hotfix/*` branch is independently created from wherever `HEAD` happened to be at that moment.

Warning: naming branches with slashes (`feature/x` , `hotfix/y`) is organizational sugar only. Don't assume merging or deleting a "parent" name affects branches that merely share its prefix.

F2 — `==` vs `startsWith()` in the workflow condition

THE MISTAKE, CAUGHT BEFORE IT SHIPPED

Proposed writing `github.ref == 'refs/heads/hotfix/'` to match any hotfix branch.

WHY IT WOULD HAVE SILENTLY FAILED

`==` requires an exact, full-string match. `'refs/heads/hotfix/'` with nothing after the final slash matches no real branch that would ever exist — every actual hotfix branch has something after that slash (`hotfix/dark-mode` , `hotfix/checkout-bug`). The condition would have evaluated to `false` for every real-world case, and the job would have simply never run — with no error, no warning, just silent non-execution.

CORRECT VERSION

`startsWith(github.ref, 'refs/heads/hotfix/')` — checks a prefix, matching the whole family of branch names regardless of what follows.

Warning: a mismatched `if:` condition in GitHub Actions doesn't error — it just skips the job quietly. A "green" pipeline run can still mean an important job never actually executed. Always check the job list inside a run, not just the overall pass/fail color.

F3 — Merged into `main` locally before ever confirming the hotfix branch worked

THE SEQUENCE, EXACTLY AS IT HAPPENED

Asked "how do I test without merging to main" — then, in the very next set of commands, ran `git push -u origin hotfix/dark-mode` immediately followed by `git merge hotfix/dark-mode` while sitting on `main`, without checking the Actions tab for the hotfix branch's run first.

WHY THIS IS A REAL PROCESS MISTAKE, NOT JUST A TECHNICALITY

The entire point of testing on a branch first is to verify *before* combining it with your trunk. Merging immediately after pushing — before looking at whether CI even passed — defeats the purpose of having asked the question in the first place.

WHAT ACTUALLY PREVENTED HARM

`git merge` only updated the *local* copy of `main` — `git push` for `main` hadn't been run yet, so nothing had reached `origin/main` or triggered production's pipeline. The mistake was caught and corrected before it became consequential, but only because the local/remote distinction happened to provide a safety window — not because the process itself was safe.

Warning: a local `git merge` feels final but isn't — nothing is truly "shipped" until `git push`. Don't rely on this gap as a safety net on purpose; always explicitly verify a branch's own CI result before merging it anywhere, rather than merging first and checking second.

F4 — "Why did it not show the two commits I made?"

THE CONFUSION

Two separate local commits were made (the feature, then the CI change) but only one workflow run appeared for the push.

NOT A BUG

GitHub Actions triggers once per *push event*, not once per commit contained within that push. Both commits existed locally before the single `git push` was run — so exactly one push event occurred, and exactly one workflow run resulted, covering the combined state of both commits together.

PATTERN

Batching multiple related commits into a single push is completely normal and does not "lose" any of them — CI reflects the branch's current tip state, not a replay of individual commits.

F5 — False alarm: "the button isn't on staging" (first investigation)

SYMPTOM

After merging locally, the toggle button appeared to be missing when checked on staging.

FIRST ROUND OF INVESTIGATION

Requested a second Claude-Code prompt specifically to "fix" the button's visibility. It reported success and testing confirmed it worked **on localhost**.

THIS IS WHERE IT GETS GENUINELY CONFUSING, AND WORTH WALKING THROUGH EXACTLY

After the "fix" prompt, `git status` reported **"nothing to commit, working tree clean."** Investigation via `grep -r "theme-toggle\|toggleTheme"` confirmed the button markup already existed in all four view files. Cross-checking `git log` showed the button had, in fact, already been present since the very *first* dark-mode commit (`374cb1b`) — the "fix" prompt found nothing wrong to fix and made no changes, which is exactly why there was nothing new to commit.

PATTERN

"Nothing to commit" after asking an AI tool to fix something can mean two very different things: (1) the fix already existed and nothing needed changing, or (2) the tool silently failed to make any change at all. These look identical in `git status` and must be told apart by independently checking the actual file contents (`grep / cat`), not just trusting a tool's own "success" report.

F6 — The real bug: staging's deploy script hardcoded `main` regardless of trigger branch

SYMPTOM

CI showed green for the `hotfix/dark-mode` branch, but SSHing directly into the staging server and running `git log --oneline -3` showed it was still sitting on an old commit from *before* the dark-mode work — two commits behind.

HOW IT WAS ACTUALLY FOUND

Rather than trusting the green pipeline status, connected directly to the server itself and asked git the ground truth. This is the same "don't trust the layer above, verify the layer itself" discipline from the very first nginx incidents tonight.

ROOT CAUSE — THE REAL ONE, HIDING UNDER THE TWO FALSE ALARMS ABOVE

The `deploy-staging` job's script contained a hardcoded line: `git pull origin main`. This ran *regardless of which branch actually triggered the deploy*. The `startsWith` condition correctly let the job *execute* on a hotfix branch — but nothing inside the script itself knew or cared which branch that was; it always, unconditionally, pulled `main`. Since a hotfix branch's whole point is to *not yet be on main*, this meant hotfix deploys to staging were silently deploying stale, wrong code every time — a real, structural gap in the pipeline design itself, not a one-off mistake.

Warning — this is the single most important lesson in this entire saga: a pipeline job can be perfectly triggered (correct `if:` condition, correct branch, green checkmark) and still deploy the *wrong code*, if the deploy script itself doesn't dynamically account for which branch triggered it. A green CI run proves the *job ran* — it does not prove the job did the *correct thing*. Always independently verify what's actually running on a server after a multi-branch pipeline change, don't infer it from pipeline color alone.

THE CORRECTED SCRIPT

```
script: |
  cd ~/northwind-design-studio-staging
  git fetch origin
  git checkout ${GITHUB_REF_NAME}
  git pull origin ${GITHUB_REF_NAME}
  npm install
  pm2 restart northwind-staging || pm2 start server.js --name northwind-staging
```

`git fetch origin` downloads information about all remote branches without changing any files yet — needed because the server may never have heard of a newly created branch. `git checkout ${GITHUB_REF_NAME}` — `GITHUB_REF_NAME` is a built-in GitHub Actions variable holding the actual branch name that triggered *this specific run*; the server switches its working files to that branch instead of assuming. `git pull origin ${GITHUB_REF_NAME}` then pulls the latest commits for whichever branch that turned out to be. Production's `deploy` job was deliberately left hardcoded to `main`, since production should only ever deploy from there — the dynamic version was only needed for staging, which now legitimately serves multiple branches.

F7 — `git revert` blocked by uncommitted local changes

SYMPTOM

Attempting to revert a commit failed: `error: Your local changes to the following files would be overwritten by merge ... Aborting.`

GENERAL MEANING OF THIS ERROR

Git refuses to run an operation that would silently discard uncommitted work sitting in the working directory. This is a protective failure, not a destructive one — git stopped itself specifically to avoid losing something.

THE RESOLUTION PATTERN

`git status` and `git diff` first, always, to see exactly what the pending uncommitted changes actually are, before deciding whether to `git stash` them out of the way and retry, or whether those changes were themselves already the intended fix and simply need to be committed directly instead of also reverting on top of them.

Warning: when git blocks an operation citing "local changes would be overwritten," resist the urge to force past it. The specific content of those changes determines the correct next step, and guessing wrong risks silently combining or losing work. Always read `git status` / `git diff` before choosing `stash` vs. `commit` vs. `discard`.

F8 — Why `git pull` before `git merge`, every time

THE QUESTION

Whether pulling before merging into `main` was actually necessary.

SAGA OUTCOME: THE DARK-MODE TOGGLE SHIPPED SUCCESSFULLY TO BOTH STAGING AND PRODUCTION, THROUGH THE NEWLY BUILT HOTFIX PATH AND THE STANDARD TRUNK-BASED PATH RESPECTIVELY. THE REAL LASTING VALUE OF THIS SAGA WASN'T THE FEATURE — IT WAS PROVING, THE HARD WAY, THAT A MULTI-BRANCH PIPELINE NEEDS EVERY SCRIPT INSIDE IT TO BE BRANCH-AWARE, NOT JUST ITS TRIGGER CONDITIONS.

G. Merge Conflict Resolution (Deliberate Drill)

Unlike every other entry in this log, nothing here was a mistake — this was a controlled, deliberately staged scenario, built on purpose to safely experience something that hadn't been touched yet: two people editing the same line of the same file at the same time. Documented in full because the mechanics are genuinely confusing the first time, no matter how it's explained in the

abstract.

G1 — Setting up the collision on purpose

THE SCENARIO

Simulating two designers both editing the homepage hero tagline in the same week.

STEPS TAKEN, IN ORDER

```
git checkout -b tagline-update
# edited the tagline line in views/index.ejs, own wording
git commit -m "... " # committed locally, NOT pushed yet

git checkout main
# edited the SAME line in views/index.ejs, different wording
git commit -m "... "
git push # pushed straight to main, playing the "teammate" role

git checkout tagline-update
git merge main
```

WHY THIS SETUP ACTUALLY REPRODUCES A REAL COLLISION

Both branches started from the exact same commit (the "fork point"). From there, each branch independently gained one new commit touching the identical line, with different content. This is precisely how two real developers collide — neither did anything wrong individually; the collision only exists because their timelines briefly diverged on the same piece of code.

G2 — What git actually printed, and what each line means

```
Auto-merging views/index.ejs
CONFLICT (content): Merge conflict in views/index.ejs
Automatic merge failed; fix conflicts and then commit the result.
```

"AUTO-MERGING VIEWS/INDEX.EJS"

Git always attempts an automatic merge first, on every file, before giving up. This line simply confirms it tried on this file — it does not mean it succeeded.

"CONFLICT (CONTENT)"

Git found two different versions of the same line(s) in the same file and has no rule for choosing between them automatically. This is git being *cautious*, not broken — it refuses to silently pick a winner and discard the other person's work.

"FIX CONFLICTS AND THEN COMMIT THE RESULT"

Git is explicitly telling you the two-step process: resolve the file by hand, then commit — it will not proceed on its own.

Reassurance, not just a warning: at this point, nothing is lost, nothing is corrupted, and nothing is permanent yet. Every file outside the conflicted one merged automatically and correctly. If it ever feels like it's going wrong, `git merge --abort` immediately cancels the entire merge attempt and returns to the exact state before it started — a full, safe undo, available at any point before the final commit.

G3 — What the conflicted file actually contains

Opening the file mid-conflict shows something like this, inserted directly into the normal code:

```
<<<<<<< HEAD
<h1>Your version of the tagline</h1>
=====
<h1>Teammate's version of the tagline</h1>
>>>>>> main
```

<<<<<<< HEAD

Marks the start of **your current branch's** version — `HEAD` means "whichever branch you have currently checked out," which was `tagline-update` in this drill. Everything from this marker down to the next one is exactly what your branch had.

=====

The dividing line — purely a visual separator between the two competing versions. Carries no meaning of its own beyond "here's where one version ends and the other begins."

>>>>>> MAIN

Marks the end of the **incoming** branch's version — in this case `main`, since that's what was being merged *in*. Everything between `=====` and this marker is what the other branch had.

THE CORE IDEA

Git did not guess, did not blend, and did not silently prefer one side. It pasted *both* full answers directly into the file and is waiting for a human decision — keep one, keep the other, or write a third version combining both.

G4 — Resolving it, and which branch it happens on

THE STEPS

```
git status          # confirms: On branch tagline-update, Unmerged paths
# open the file, delete all three marker lines,
# keep/combine the actual wording you want
git add views/index.ejs
git commit          # opens an editor with a pre-filled merge message — save as-is
```

WHICH BRANCH DOES THIS LAND ON?

Whichever branch was checked out when `git merge` was run — in this drill, `tagline-update`, since that's where the `git checkout tagline-update` then `git merge main` sequence happened. The resulting merge commit lives on `tagline-update` only, combining both histories into one branch. `main` itself is untouched by this process — it only changes later, when `tagline-update` is eventually merged back into it, same as any other feature branch.

NO -M FLAG ON THIS COMMIT — WHY

Git already knows this is a merge commit and pre-fills a sensible default message (something like "Merge branch 'main' into tagline-update") in an editor. Accepting the default is normal and expected; there's no requirement to write a custom message the way a regular commit needs one.

G5 — Clarification: does `git revert` require a commit to be pushed first?

SHORT ANSWER

No. `revert` works on **any** commit that exists in local history, pushed or not — what matters is that the commit has a real hash to point to, not whether it's reached GitHub yet.

WHERE THE CONFUSION LIKELY COMES FROM

`revert` is specifically the *safe* choice once something has been pushed and others may have already pulled it — because it adds a new commit rather than rewriting history, nobody else's copy gets disrupted. `reset`, by contrast, rewrites history, which becomes genuinely dangerous only after a push, once other people's local copies could conflict with the rewritten timeline. The distinction isn't "revert needs a push" — it's "revert remains safe regardless of push status, while reset's danger specifically depends on it."

H. Deliberate Drills (Practiced, Not Broken by Accident)

Two more scenarios were deliberately staged, not bugs — same spirit as the merge-conflict drill, documented for the same reason: the mechanics only make sense once actually run, not just described.

H1 — Git Stash Drill: the "urgent interruption" scenario

THE SCENARIO

Mid-edit on a file, an urgent unrelated task arrives — need to set the in-progress work aside without committing half-finished changes, then come back to it.

STEPS RUN, IN ORDER

```
echo "<!-- testing stash -->" >> views/index.ejs
git status          # shows the file as modified
git stash           # shelves the change, no commit, no message
git status          # working directory now clean again
git stash list       # confirms the change is saved on a "shelf"
git stash pop        # re-applies the change AND removes it from the shelf
git status           # the edit is back exactly as it was
git checkout -- views/index.ejs # cleanup: discard the test line entirely
```

WHY STASH SPECIFICALLY, OVER THE ALTERNATIVE OF COMMITTING TO A THROWAWAY BRANCH

A WIP (work-in-progress) commit on a side branch works too, but it's heavier — a permanent, half-finished entry in project history, plus a full branch/push/pull round-trip just to set something aside temporarily. `stash` has no message, no history entry, no branch — it's built specifically for "I'll be right back."

NOTE ON STASH POP VS STASH APPLY

`pop` re-applies the change and removes it from the stash list in one step. `apply` re-applies but leaves it on the shelf too — useful if the same stashed change might be needed again on a different branch.

NOTE ON GIT CHECKOUT -- <FILE>

Discards uncommitted changes to a specific file, reverting it to the last commit's version — a hard, silent discard with no undo,

unlike `revert`. The `--` tells git "everything after this is a file path, not a branch name," resolving the ambiguity between `git checkout`'s two different jobs (switching branches vs. restoring a file). Modern git offers a clearer, purpose-built replacement: `git restore <file>` — same effect, less ambiguous name, no `--` needed.

H2 — Rollback Drill: proving it actually works, not just describing it

THE SCENARIO

The tracker task literally requires *testing* the rollback path, not just designing it — so a real, breakable change was shipped through the full pipeline on purpose, then undone.

STEPS RUN, IN ORDER

```
git checkout -b test/rollback-drill
# made an obviously-wrong change, e.g. a broken heading — committed as "purposeful-error"
git push -u origin test/rollback-drill
# confirmed on the Actions tab: test + deploy-staging both ran, staging visibly showed the broken version
git log --oneline                # located the exact bad commit's hash
git revert <purposeful-error-hash>
git push
# confirmed: pipeline ran again, staging now showed the corrected version
```

WHAT THIS ACTUALLY PROVED

That a rollback isn't a special, separate mechanism — it's the exact same pipeline, triggered by the exact same kind of push, just carrying a commit whose job is "undo," instead of "add." No shortcut, no bypass — the safety net (tests, staging, approval-gated production) applies to a rollback exactly as it applies to any other change.

5. Decisions & Trade-offs Considered

Not every decision tonight resulted in code. Some of the most valuable thinking happened in choices that were deliberately *not* taken — documented here because the reasoning matters as much as anything that got built.

5.1 — Real company logos vs. fictional ones (declined)

Early in the design work, real company logos (including a specific request to depict a named real company as a "client") were requested for a "Partners" section, then insisted on a second time with the framing "it's for practice." This was declined both times. Reasoning: this repo is portfolio material meant to be shown to real employers — a fictional agency listing real, named companies as clients reads as either a false claim or a credibility-damaging mistake, not as harmless practice. It also involves reproducing trademarked logo designs. The resolution: fictional client logos (the simulator's own other companies — Milk & Bean, CrateBox, etc.) for the "clients" section, plus a legitimate "Built with" strip of real technology names (AWS, Node.js, Docker) — a true claim about the stack, not a false claim about relationships.

5.2 — Pixel-perfect clone of a real photographer's website (declined)

A detailed extraction spec (exact colors, fonts, layout, real contact links) for a real, living photographer's personal site was provided with a request to clone it exactly for Northwind's redesign. Declined — this would reproduce a real person's actual creative work, content, and identity, not just draw stylistic inspiration. The resolution: extracting the underlying *aesthetic direction* (editorial, paper-toned palette, minimal typography-led layout) into an original prompt, with entirely Northwind-owned content, placeholder imagery, and no real names or contact details anywhere.

5.3 — Cart storage: SQLite vs. RDS vs. DynamoDB (parked, not yet built)

When a shopping-cart feature was proposed, three real storage options were weighed: **SQLite** (a real relational database, but just a file — zero networking/VPC setup required, matching the skill level at this point in the program, since VPC/security-group concepts hadn't been taught yet); **RDS** (the AWS-managed option explicitly named in the program's target job spec, but "correct" setup needs private subnets and tight security groups — concepts reserved for a later project); **DynamoDB** (AWS's serverless NoSQL option, no VPC required at all, a natural shape-fit for a cart's simple per-user record). The explicit choice made: RDS, deliberately accepting a "quick, publicly-accessible now, properly secured later" trade-off — mirroring the same pattern already used for the EC2 root-user lesson elsewhere in this program. **As of this document, RDS/the cart feature itself was deliberately parked and not yet built** — the decision was made, the reasoning is recorded, but the implementation was consciously deferred.

5.4 — Facebook OAuth: removed after initial inclusion

Facebook login was included in the original OAuth scaffolding prompt alongside Google, but was later removed entirely — the Google flow was fully working and tested; Facebook was left unfinished/untested and judged not worth shipping in a half-working state. A targeted follow-up prompt removed the Facebook strategy config, its routes, any frontend login button referencing it, related tests, and its env var references from `.env.example` — confirmed as a clean removal (Google and Stripe checkout re-tested afterward to ensure nothing else broke in the process).

6. Command Reference

Git

Command	Purpose
<code>git status</code>	What's changed since the last commit
<code>git add <file> / git add .</code>	Stage specific file(s) or everything
<code>git commit -m "..."</code>	Save a snapshot locally
<code>git push / git push -u origin <branch></code>	Upload commits; <code>-u</code> sets tracking on first push of a new branch
<code>git pull</code>	Download and merge remote changes
<code>git log --oneline</code>	Compact commit history with short hashes
<code>git checkout <hash></code>	View an old commit (detached HEAD, safe)
<code>git checkout main</code>	Return to the current tip of main
<code>git checkout -b <name></code>	Create and switch to a new branch
<code>git branch -m <old> <new></code>	Rename a branch
<code>git stash / git stash pop</code>	Shelve uncommitted changes / bring them back
<code>git stash list</code>	See everything currently shelved
<code>git revert <hash></code>	Safely undo one commit via a new commit — history preserved
<code>git restore <file></code>	Discard uncommitted changes to a file (modern replacement for <code>checkout -- file</code>)
<code>git diff</code>	Line-by-line view of unstaged changes
<code>git merge <branch></code>	Merge a branch into the current one

GitHub CLI

Command	Purpose
<code>gh auth login</code>	One-time authentication
<code>gh secret set NAME --body "value"</code>	Set a short secret value directly
<code>gh secret set NAME < file.pem</code>	Set a secret from a file's exact contents (safer for keys)
<code>gh secret list</code>	Confirm which secrets exist (values stay hidden)
<code>gh run list --limit N</code>	Recent workflow runs and their status
<code>gh run cancel <id></code>	Cancel a specific run
<code>gh run view <id></code>	See a run's true current status

Server / Deploy

Command	Purpose
<code>rsync -avz -e "ssh -i key.pem" --exclude 'node_modules' src/ user@host:dest/</code>	Copy files up while excluding heavy/unneeded folders
<code>sudo nginx -t</code>	Validate nginx config syntax before reloading
<code>sudo systemctl reload nginx</code>	Apply a config change without dropping connections
<code>sudo certbot --nginx -d domain.com</code>	Issue/attach an HTTPS cert to a matching nginx block
<code>pm2 start server.js --name X</code>	Run an app in the background under a specific name
<code>pm2 restart X</code>	Restart an existing named process

<code>pm2 list / pm2 logs X --lines N --nostream</code>	See running processes / read a process's real error output
<code>pm2 save / pm2 startup</code>	Persist the process list / register pm2 as a boot-time service
<code>curl -I http://host / curl -Ik https://host</code>	Test a URL from the command line, headers only (<code>-k</code> skips cert validation)
<code>ss -tlnp grep <port></code>	Confirm what's actually listening on a given port

7. Configuration Files — Every Line Explained

The incident log explains what went wrong. This section explains what's actually sitting on disk right now, line by line, independent of any bug — the reference to come back to whenever a new project's config looks unfamiliar.

7.1 — Nginx: Staging Site Config (full file, current state)

Path: `/etc/nginx/sites-available/staging.northwind-design.online`

```
server {
    if ($host = staging.northwind-design.online) {
        return 301 https://$host$request_uri;
    }

    listen 80;
    listen [::]:80;
    server_name staging.northwind-design.online;
    return 404;
}

server {
    listen 443 ssl;
    server_name staging.northwind-design.online;

    ssl_certificate /etc/letsencrypt/live/staging.northwind-design.online/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/staging.northwind-design.online/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    location / {
        auth_basic "Northwind Staging - Restricted";
        auth_basic_user_file /etc/nginx/.htpasswd;

        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

FIRST BLOCK — THE HTTP (PORT 80) REDIRECT, WRITTEN BY CERTBOT

`if ($host = staging.northwind-design.online) { return 301 https://$host$request_uri; }` — checks the domain the browser actually asked for; if it matches, sends back HTTP status **301** (a permanent redirect) to the exact same path, but with `https://` instead. `$host` and `$request_uri` are nginx variables holding the requested domain and the requested path — this line works for any path on the site, not just the homepage, without hardcoding anything.

`listen 80;` and `listen [::]:80;` — the first handles regular IPv4 traffic on port 80, the second handles IPv6 traffic on the same port. Both are needed for full coverage.

`server_name staging.northwind-design.online;` — same purpose as always: this block only applies to requests for this exact domain.

`return 404;` — a fallback. If somehow a request reaches this block without matching the `if` above (edge case, rare), nginx returns "not found" rather than silently doing nothing.

SECOND BLOCK — THE ACTUAL HTTPS (443) SITE

`listen 443 ssl;` — this block handles encrypted HTTPS traffic specifically; `ssl` tells nginx to expect and terminate TLS on this port.

`ssl_certificate /` `ssl_certificate_key` — the public certificate and the private key Let's Encrypt issued, both stored under `/etc/letsencrypt/live/<domain>/`. The certificate proves identity to visiting browsers; the key is what makes the encryption actually work and must never be shared or committed anywhere.

`include /etc/letsencrypt/options-ssl-nginx.conf;` — pulls in a separate file (maintained by certbot) containing Let's Encrypt's current recommended secure TLS settings — which protocol versions and ciphers to allow — so this file doesn't have to hardcode security best-practices that change over time.

`ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;` — a separate cryptographic parameter file that strengthens the key-exchange step of the TLS handshake. Generated once by certbot, reused by every HTTPS site on the box.

`location / { ... }` — same meaning as always: applies to every path.

`auth_basic "Northwind Staging - Restricted";` — turns on HTTP basic authentication for everything inside this `location` block. The quoted text is the label shown in the browser's native login prompt — purely cosmetic, doesn't affect function.

`auth_basic_user_file /etc/nginx/.htpasswd;` — points nginx at the password file generated by `htpasswd`. Without this line, `auth_basic` alone does nothing — nginx needs to know *which* credentials are valid.

`proxy_pass` / `proxy_set_header Host` / `proxy_set_header X-Real-IP` — unchanged from the very first reverse-proxy lesson: forward the (now-authenticated) request to the Node app on port 3000, passing the real domain and visitor IP through.

7.2 — Nginx: Production Site Config

Path: `/etc/nginx/sites-available/northwind-design.online` — identical shape to staging, two real differences worth naming:

- `server_name` lists **two** domains — `northwind-design.online` `www.northwind-design.online` — since production serves both the bare domain and the `www` version, unlike staging which only has one hostname.
- **No `auth_basic` lines** — production is meant to be publicly reachable by real users; the password gate only belongs on staging.

7.3 — GitHub Actions: Full Workflow File

Path: `.github/workflows/ci.yml`

```
name: CI
on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm install
      - run: npm test

  deploy-staging:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy to Staging
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.STAGING_HOST }
          username: ${ secrets.STAGING_USER }
          key: ${ secrets.STAGING_SSH_KEY }
          script: |
            cd ~/northwind-design-studio-staging
            git pull origin main
            npm install
            pm2 restart northwind-staging || pm2 start server.js --name northwind-staging

  deploy:
    needs: deploy-staging
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    environment:
      name: production
    steps:
      - name: Deploy to Production
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd ~/northwind-design-studio
            git pull origin main
            npm install
            pm2 restart northwind || pm2 start server.js --name northwind
```

`name: CI` — cosmetic label, shown in the Actions tab UI only.

`on: push` — triggers this workflow on a push to **any** branch. Deliberately broad at the top level; individual jobs narrow it further with `if: .`

`jobs:` — container for everything below; each child key (`test` , `deploy-staging` , `deploy`) is a separately-named job.

`test:` job — `runs-on: ubuntu-latest` spins up a brand-new, empty virtual machine. `actions/checkout@v4` pulls the repo's code onto that empty machine — nothing exists there otherwise. `actions/setup-node@v4` with `node-version: '22'` installs Node (quoted, so YAML doesn't misread it as a number). `npm install` and `npm test` run exactly as they would locally — if `npm test` exits with a failure, this whole job is marked red and nothing downstream runs.

`deploy-staging`: job — `needs: test` means this job waits and only runs if `test` succeeded. `if: github.ref == 'refs/heads/main'` restricts it to pushes on `main` specifically — feature branches still run `test`, but never trigger a deploy. `uses: appleboy/ssh-action@v1` is a community-published action whose entire job is: SSH into a server and run commands. `host / username / key` are pulled from GitHub Secrets — never appear in plaintext, never shown in logs even on failure. `script: |` — the pipe means "everything indented below is a multi-line block, run as-is"; each line executes **on the remote server**, in order: move into the app folder, pull the latest code, reinstall dependencies (in case anything changed), then restart the running process — falling back to starting it fresh if it isn't already running (the `||` operator: "if the left side fails, run the right side instead").

`deploy`: job — identical shape to `deploy-staging`, three differences: it targets production's own secrets (`EC2_HOST / EC2_USER / EC2_SSH_KEY`, separate from staging's), it uses `needs: deploy-staging` so it only runs after staging succeeds, and the `environment: name: production` block ties it to a protection rule configured separately in the repo's Settings — this is what makes the job pause and wait for a human's manual approval before it actually executes, rather than running immediately like the other two jobs.

7.4 — .gitignore

```
node_modules/
.env
.env.*
npm-debug.log*
coverage/
.DS_Store
!.env.example
```

Each line is a pattern git will refuse to track. `node_modules/` — dependencies, regenerated by `npm install` anywhere, never worth committing. `.env` — real secrets, must never enter git history. `.env.*` — a wildcard catching any file starting with `.env.`, meant to catch environment-specific variants (`.env.production`, `.env.local`, etc.) — but broad enough that it accidentally also matched `.env.example`. `npm-debug.log*` — crash logs npm sometimes generates, purely local noise. `coverage/` — generated test-coverage reports, regenerable, not source. `.DS_Store` — a macOS Finder metadata file, irrelevant to the project and irrelevant on other operating systems. `!.env.example` — the negation fix: the `!` carves out an explicit exception to the `.env.*` rule above it, so this one specific, secret-free template file *is* tracked, letting anyone see which variable names the app expects without ever seeing real values.

7.5 — .env (structure only — variable names, never real values)

Variable	Purpose
<code>APP_BASE_URL</code>	The app's own bare domain, used to build every OAuth/Stripe callback URL correctly
<code>SESSION_SECRET</code>	Random string signing session cookies, so nobody can forge a fake "logged in" cookie — generated fresh per environment via <code>openssl rand -hex 32</code>
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	Credentials from the registered Google Cloud OAuth app — separate pairs for staging vs. production
<code>STRIPE_SECRET_KEY</code> / <code>STRIPE_PUBLISHABLE_KEY</code>	Test-mode Stripe API keys powering the checkout flow — never live/real payment keys in this project

None of these values ever appear in git — the file itself is gitignored, and on each server it was created manually, once, by hand, after every fresh clone or deploy setup.

ON QUOTING VALUES

`.env` files are plain `KEY=value` — `dotenv` reads everything after `=` literally. For values with no spaces or special characters (a URL, a hex string), quotes are unnecessary and actively harmful — `SESSION_SECRET="abc123"` would make the quote marks themselves part of the secret, not just visual decoration. Quotes are only needed if a value itself contains a space, e.g. `APP_NAME="Northwind Design Studio"` — none of the values in this project's `.env` needed them.

7.6 — package.json test script

```
"scripts": {
  "test": "jest"
}
```

This is the entire reason `npm test` works — it's an alias, not a built-in feature of npm. Running `npm test` literally means "look up the string next to `"test"` in this file and execute that." Here it points at **Jest**, the actual test-running package (a separate, real dependency in `package.json`) that finds test files, runs their assertions, and reports pass/fail.

8. Modern Alternatives & Perspective

Everything built tonight is real, industry-standard, and still widely used — none of it is "wrong." But it's also deliberately the **manual, fundamentals-first** version of each tool, on purpose, so the underlying mechanics are actually understood rather than hidden behind abstraction. Worth knowing what a more modern, "just make it work faster" version of the same stack looks like — not to redo tonight's work, just for perspective on where the industry has moved toward, and why the harder path taken here still has real value.

Nginx + Certbot → Caddy

What was built: a separate reverse-proxy config, a separate certbot install and run, certbot rewriting the config to add HTTPS.

The modern alternative: [Caddy](#), a newer web server that gets automatic HTTPS essentially for free — a config as short as `northwind-design.online { reverse_proxy localhost:3000 }` handles the proxy *and* requests/renews a cert automatically, no separate certbot step at all. **Why nginx was still worth learning first:** nginx is still the dominant choice in real production infrastructure, and understanding manually what Caddy does automatically (privileged ports, the redirect, the renewal timer) means never being confused when something in an automated system breaks.

Raw EC2 + pm2 → Containers, or a managed platform

What was built: a bare Ubuntu server, Node installed directly on it, `pm2` keeping the process alive. **The modern alternative:** package the app as a Docker container (covered later, in project 04) and run it via ECS, EKS, or a simpler managed container platform — or skip infrastructure management almost entirely with a platform like Render, Railway, or Fly.io, which handle process supervision, HTTPS, and scaling without ever touching a raw EC2 instance or writing an nginx config by hand. **Why the raw version still matters:** managed platforms are themselves built on exactly these primitives (a process manager, a reverse proxy, a cert authority) — knowing what's underneath means never being helpless when the abstraction leaks or the platform doesn't support something needed.

Manual AWS Console Work → Terraform

What was built: every AWS resource (EC2 instances, security groups, Route 53 records) clicked into existence by hand, in the console. **The modern alternative:** define all of it as code (Terraform, CloudFormation, or AWS CDK) and run `terraform apply` — repeatable, reviewable, versioned in git alongside the app itself. **This isn't skipped forever** — it's explicitly project 06 in this same program (Groundwork Builders), taught only after enough manual AWS experience exists to actually understand what the generated code is doing, rather than copy-pasting Terraform without a mental model underneath it.

Long-Lived SSH Keys in GitHub Secrets → OIDC / Short-Lived Credentials

What was built: a permanent SSH private key, stored as a GitHub Secret, used by every deploy indefinitely until manually rotated. **The modern alternative:** GitHub Actions supports OpenID Connect (OIDC) federation directly with AWS — the pipeline requests a short-lived, auto-expiring credential at runtime instead of holding a permanent secret at all, removing an entire category of "a leaked key works forever" risk. **Why the SSH-key version is still worth having built:** OIDC configuration is genuinely more abstract and harder to debug when it goes wrong — understanding the plain SSH-based flow first makes the more automated version legible instead of magical.

Manually-Written Deploy Scripts → GitOps / Purpose-Built CD Tools

What was built: a hand-written `script:` block inside GitHub Actions, doing `git pull` + `npm install` + `pm2 restart` over raw SSH. **The modern alternative:** tools like ArgoCD or Flux (common in Kubernetes-based setups — foreshadowing project 07) watch a git repository directly and continuously reconcile the live environment to match it, without a pipeline ever needing to SSH anywhere at all. **Why this matters less at this stage:** GitOps tools assume container/Kubernetes infrastructure that hasn't been introduced yet in this program — the SSH-script approach is the correct, honest version for where the curriculum is right now, not a mistake to fix later.

Passport.js (self-hosted OAuth) → Managed Auth Platforms

What was built: Passport.js handling the OAuth handshake, session management via `express-session`, all self-hosted logic. **The modern alternative:** managed auth platforms (Clerk, Auth0, or the newer Auth.js) handle the entire flow, session storage, and even UI components, reducing custom auth code to nearly zero. **Why building it manually still had value:** understanding what a session actually is, why a callback URL must match exactly, and what a managed platform is doing under the hood — the exact same reasoning as nginx vs. Caddy above.

The overall pattern: almost every "modern" alternative above is a more automated version of a primitive that was hand-built tonight. That's not a coincidence — it's the actual design of this program. Automation is trustworthy precisely to the degree the thing being automated is understood. Every shortcut in the list above is worth adopting eventually — just not

before the manual version has been felt at least once.

9. The Underlying Skill

Nearly every incident above was solved the same way, regardless of the specific technology: **test each layer in isolation, closest-to-the-symptom first, before assuming the cause.** App alive? Then server healthy? Then network/security group open? Then DNS resolving? Then credentials correct? Each layer ruled in or out with a real command, not a guess — and the actual error message read carefully before reacting to it, since several of tonight's bugs (the triple-slash URL, "Revert 'initial'", the EADDRINUSE trace) named their own root cause directly, if read closely rather than skimmed.

This is the same discipline project 03 (Roadrunner Logistics) will deliberately drill again — tonight was real, unplanned practice of it, under actual production conditions rather than a staged exercise.